

LAB2 - Programação para Sistemas Paralelos e Distribuídos T02

Estudo sobre Hadoop e Spark

ANA LUIZA RODRIGUES DA SILVA - 211030676

GUILHERME BRITO VILAS BOAS - 190108011

LAÍS RAMOS BARBOSA - 170107574

LUCAS LOPES ROCHA - 202023903

MATHEUS RAPHAEL SOARES DE OLIVEIRA - 190058587

Contents

1	Introdução	2
1.1	Visão Geral do Documento	3
2	Conhecendo o Apache Hadoop	3
2.1	Montagem de um cluster Hadoop básico	3
2.1.1	Arquitetura do cluster	3
2.1.2	Containers	4
2.2	Estrutura de arquivos	4
2.3	Configurações principais	4
2.3.1	HDFS (Hadoop Distributed File System)	4
2.3.2	YARN (Resource Manager)	4
2.3.3	MapReduce	5
2.4	Interfaces Web	5
2.4.1	1. Namenode Web UI	5
2.4.2	2. ResourceManager Web UI	5
2.5	Execução	5
2.5.1	Executando Word Count (Python)	6
2.5.2	Executando Word Count (Java)	8
2.6	Materiais utilizados de base	9
2.6.1	Configurações Implementadas	9
2.7	Teste de Tolerância a faltas e performance de aplicações Hadoop	18
2.7.1	Teste 1: Baseline	19
2.7.2	Teste 2: Falha pré-existente	22
2.7.3	Teste 3: Falha em tempo real	30
2.7.4	Conclusão	40

3	Conhecendo o Apache Spark	41
3.1	Visão Geral da Implementação	41
3.2	Componentes e Tecnologias	41
3.2.1	1. Apache Spark Streaming	41
3.2.2	2. Apache Kafka	41
3.2.3	3. Reddit Scraper	41
3.2.4	4. Análise de Sentimentos (VADER)	42
3.2.5	5. Elasticsearch	42
3.2.6	6. Consumer Kafka-to-ElasticSearch	42
3.2.7	7. Kibana	43
3.3	Visualizações Implementadas	43
3.3.1	1. Nuvem de Palavras (Tag Cloud)	43
3.3.2	2. Distribuição de Sentimentos (Gráfico de Pizza)	43
3.3.3	3. Contador de Palavras (Gráfico de Barras Horizontal)	45
3.3.4	4. Sentimentos ao Longo do Tempo (Gráfico de Barras)	45
3.3.5	5. Dashboard Completo	46
3.4	Como Executar a Aplicação	46
3.4.1	Pré-requisitos	46
3.4.2	Primeira Execução - Passo a Passo Detalhado	46
3.4.3	Execuções Subsequentes	48
3.4.4	Parar a Aplicação	49
3.5	Conclusão sobre o Uso do Spark	49
3.5.1	Dificuldades encontradas	49
4	Autoavaliação	50

1 Introdução

Este documento apresenta o desenvolvimento e a documentação do Laboratório 2 da disciplina de Programação para Sistemas Paralelos e Distribuídos (PSPD), que tem como objetivo principal o estudo prático e aprofundado de duas das principais tecnologias de processamento distribuído de dados: **Apache Hadoop** e **Apache Spark**.

O trabalho foi estruturado em duas grandes etapas. Na primeira etapa, foi solicitada a implementação de um cluster Hadoop funcional, incluindo a configuração de um sistema de arquivos distribuído (HDFS), o gerenciamento de recursos com YARN, e a execução de aplicações MapReduce. Além disso, foi necessário realizar testes de tolerância a falhas e otimização de performance, explorando diferentes configurações de alocação de recursos, escalonamento de tarefas e distribuição de blocos. Na segunda etapa, o foco foi direcionado ao Apache Spark, onde foi desenvolvida uma aplicação de streaming em tempo real integrada com Apache Kafka, Elasticsearch e Kibana, realizando análise de sentimentos de dados coletados do Reddit.

Clique aqui para acessar o vídeo de apresentação

[Clique aqui para acessar o repositório no github](#)

1.1 Visão Geral do Documento

Este documento está organizado da seguinte forma:

- **Conhecendo o Apache Hadoop:** Apresenta a implementação completa do cluster Hadoop, incluindo arquitetura, configurações, execução de aplicações MapReduce (WordCount em Python e Java), e testes detalhados de tolerância a falhas com análise de performance em diferentes cenários de falha de nós.
- **Conhecendo o Apache Spark:** Descreve a implementação de uma aplicação de streaming com Spark Structured Streaming, detalhando a arquitetura completa que envolve coleta de dados do Reddit, processamento em tempo real, análise de sentimentos com VADER, armazenamento em ElasticSearch e visualização em Kibana.

Cada seção contém instruções detalhadas de configuração, execução e análise de resultados, permitindo a reprodução completa dos experimentos realizados.

2 Conhecendo o Apache Hadoop

2.1 Montagem de um cluster Hadoop básico

Este repositório contém a implementação completa de um cluster Hadoop básico usando containers Docker, incluindo:

- 1 nó master (Namenode)
- 2 nós slaves (Datanodes)
- HDFS configurado e testado
- Interfaces Web funcionais (Namenode, ResourceManager)
- Exemplo WordCount

2.1.1 Arquitetura do cluster

```
1
2
3      MASTER NODE (Namenode)
4  - Gerencia metadados do HDFS
5  - Web UI: http://localhost:9870
6  - ResourceManager (YARN): http://localhost:8088
7
8
9
10
11
```

12	SLAVE NODE 1	SLAVE NODE 2
13	(Datanode1)	(Datanode2)
14	- DataNode	- DataNode
15	- NodeMgr	- NodeMgr

2.1.2 Containers

namenode: Master node + ResourceManager **datanode1 e datanode2:** Workers para HDFS **nodemanager1 e nodemanager2:** Workers para YARN **resourcemanager:** Gerenciador de recursos **historyserver:** Histórico de jobs

2.2 Estrutura de arquivos

```

1 hadoop-cluster/
2 docker-compose.yml           # Definição dos serviços
3 hadoop.env                   # Configurações Hadoop
4 scripts/
5     mapper.py                 # Mapper para WordCount
6     reducer.py                # Reducer para WordCount
7     WordCount.java            # Versão Java
8 dados/
9     teste.txt                 # Arquivo de teste

```

2.3 Configurações principais

2.3.1 HDFS (Hadoop Distributed File System)

Parâmetro	Valor	Descrição
fs.defaultFS	hdfs://namenode:9000	URI do filesystem
dfs.replication	2	Fator de replicação
dfs.namenode.name.dir	/hadoop/dfs/name	Dir do Namenode
dfs.datanode.data.dir	/hadoop/dfs/data	Dir dos Datanodes

2.3.2 YARN (Resource Manager)

Parâmetro	Valor	Descrição
yarn.resourcemanager.hostname	resourcemanager	Hostname do RM
yarn.nodemanager.resource.memory-mb	4096	RAM por NodeManager
yarn.nodemanager.resource.cpu-vcores	4	CPUs por NodeManager

2.3.3 MapReduce

Parâmetro	Valor	Descrição
<code>mapreduce.framework.name</code>	<code>yarn</code>	Framework usado
<code>mapreduce.map.memory.mb</code>	<code>512</code>	RAM para Map tasks
<code>mapreduce.reduce.memory.mb</code>	<code>512</code>	RAM para Reduce tasks

2.4 Interfaces Web

2.4.1 1. Namenode Web UI

- **URL:** `http://localhost:9870`
- **Funcionalidades:**
 - Status do HDFS
 - DataNodes ativos
 - Uso de espaço
 - Explorador de arquivos
 - Logs do sistema

Como acessar:

1. Abra o navegador
2. Acesse `http://localhost:9870`

2.4.2 2. ResourceManager Web UI

- **URL:** `http://localhost:8088`
- **Funcionalidades:**
 - Aplicações em execução
 - Filas de recursos
 - Jobs MapReduce
 - Uso de CPU/Memória
 - Histórico

Como monitorar um job:

1. Acesse `http://localhost:8088`
2. Clique em “Applications”
3. Clique no ApplicationID do job em execução
4. Veja progresso de Map e Reduce

2.5 Execução

```
1 cd hadoop-cluster
2 docker compose up -d
3
4 #verificar estado dos containers
5 docker ps
```

2.5.1 Executando Word Count (Python)

1. Instalar o python nos containers nodemanager1, nodemanager2, datanode1, datanode2

```
1 docker exec -it nodemanager1 bash -c "  
2 echo 'deb http://archive.debian.org/debian/ stretch main' > /etc/apt/  
  sources.list && \  
3 echo 'deb http://archive.debian.org/debian-security/ stretch/updates  
  main' >> /etc/apt/sources.list && \  
4 apt-get update && \  
5 apt-get install -y python3 && \  
6 python3 --version  
7 "
```

```
1 docker exec -it nodemanager2 bash -c "  
2 echo 'deb http://archive.debian.org/debian/ stretch main' > /etc/apt/  
  sources.list && \  
3 echo 'deb http://archive.debian.org/debian-security/ stretch/updates  
  main' >> /etc/apt/sources.list && \  
4 apt-get update && \  
5 apt-get install -y python3 && \  
6 python3 --version  
7 "
```

```
1 docker exec -it datanode1 bash -c "  
2 echo 'deb http://archive.debian.org/debian/ stretch main' > /etc/apt/  
  sources.list && \  
3 echo 'deb http://archive.debian.org/debian-security/ stretch/updates  
  main' >> /etc/apt/sources.list && \  
4 apt-get update && \  
5 apt-get install -y python3 && \  
6 python3 --version  
7 "
```

```
1 docker exec -it datanode2 bash -c "  
2 echo 'deb http://archive.debian.org/debian/ stretch main' > /etc/apt/  
  sources.list && \  
3 echo 'deb http://archive.debian.org/debian-security/ stretch/updates  
  main' >> /etc/apt/sources.list && \  
4 apt-get update && \  
5 apt-get install -y python3 && \  
6 python3 --version  
7 "
```

2. Tornar Scripts Executáveis

```
1 chmod +x scripts/*.py
```

3. Copiar Scripts para o Container

```
1 docker cp scripts/mapper.py namenode:/tmp/
2 docker cp scripts/reducer.py namenode:/tmp/
```

4. Copiar o arquivo de teste para o container

```
1 docker cp dados/teste.txt namenode:/tmp/
```

5. Criar Diretórios no HDFS

```
1 docker exec -it namenode hdfs dfs -mkdir -p /user/hadoop/input
2 docker exec -it namenode hdfs dfs -mkdir -p /user/hadoop/output
3
4 #Fazer o upload do arquivo de teste
5 docker exec -it namenode hdfs dfs -put /tmp/teste.txt /user/hadoop/
   input/
```

6. Executar WordCount com Hadoop Streaming

```
1 #Limpar o diretório de saída
2 docker exec -it namenode hdfs dfs -rm -r /user/hadoop/output 2>/dev/
   null || true
3
4 # Tornar scripts executáveis dentro do container
5 docker exec -it namenode chmod +x /tmp/mapper.py /tmp/reducer.py
6
7 docker exec -it namenode hadoop jar \
8   /opt/hadoop-3.1.3/share/hadoop/tools/lib/hadoop-streaming-3.1.3.jar
   \
9   -files /tmp/mapper.py,/tmp/reducer.py \
10  -input /user/hadoop/input/teste.txt \
11  -output /user/hadoop/output \
12  -mapper mapper.py \
13  -reducer reducer.py
```

7. Verificar resultados

```
1 docker exec -it namenode hdfs dfs -ls /user/hadoop/output/
2 docker exec -it namenode hdfs dfs -cat /user/hadoop/output/part-00000
```

- Saída esperada

```
1 Big 2
2 Data 2
3 Hadoop 3
4 MapReduce 2
5 0 3
6 framework 1
```

```
7 importante 1
8 poderoso 1
9 processa 1
10 usa 1
11 é 3
```

8. Encerrar os containers

```
1 docker compose down
```

2.5.2 Executando Word Count (Java)

1. Compilar WordCount.java

```
1 #Copiar código Java para o container
2 docker cp scripts/WordCount.java namenode:/tmp/
3
4 #Entrar no container
5 docker exec -it namenode bash
6
7 #Compilar
8 cd /tmp
9 javac -classpath `hadoop classpath` WordCount.java
10
11 # Criar JAR
12 jar cf wc.jar WordCount*.class
13
14 # Verificar
15 ls -l wc.jar
16 exit
```

2. Executar WordCount Java

```
1 #Limpar o diretório de saída
2 docker exec -it namenode hdfs dfs -rm -r /user/hadoop/output 2>/dev/
   null || true
3
4 # Executar MapReduce com Java
5 docker exec -it namenode hadoop jar /tmp/wc.jar WordCount \
6   /user/hadoop/input/teste.txt \
7   /user/hadoop/output
8
9 # Ver resultado
10 docker exec -it namenode hdfs dfs -cat /user/hadoop/output/part-r
    -00000
```

3. Encerrar os containers


```
1 docker compose down
```

2.6 Materiais utilizados de base

Notebook do professor disponível no Google Colab: UnB/ESW/PSPD
- Laboratório sobre Hadoop (HDFS e MapReduce)

Imagens Docker: big-data-europe/docker-hadoop

Para realizar testes no framework Hadoop foram editadas configurações e adicionadas novas nos arquivos de configuração `hadoop.env` e `docke-compose`.

2.6.1 Configurações Implementadas

2.6.1.1 1.Escalonamento de Tarefas (YARN)

2.6.1.1.1 Configuração 1: Fair Scheduler Arquivo: `hadoop.env`

Mudança realizada:

```
1 # padrão (Capacity Scheduler):
2 YARN_CONF yarn_resourcemanager_scheduler_class=org.apache.hadoop.yarn
  .server.resourcemanager.scheduler.capacity.CapacityScheduler
3
4 # Configuração aplicada (Fair Scheduler):
5 YARN_CONF yarn_resourcemanager_scheduler_class=org.apache.hadoop.yarn
  .server.resourcemanager.scheduler.fair.FairScheduler
```

Justificativa:

O **Capacity Scheduler** aloca recursos baseado em filas com capacidades fixas pré-definidas. Cada fila tem uma porcentagem garantida dos recursos do cluster. Este modelo é adequado para ambientes onde diferentes departamentos ou projetos precisam de garantias de recursos.

O **Fair Scheduler**, por outro lado, distribui recursos de forma dinâmica e justa entre todas as aplicações ativas. Quando uma aplicação é submetida, ela recebe uma fatia dos recursos disponíveis. Se mais aplicações chegam, os recursos são redistribuídos igualmente. Este modelo é melhor para:

- Ambientes multi-usuário
- Workloads imprevisíveis
- Redução de latência para jobs pequenos

Validação:

```
1 $ docker exec -it namenode bash -c "cat /opt/hadoop-3.1.3/etc/hadoop/
  yarn-site.xml | grep -A1 'scheduler.class'"
2
3 Resultado:
```

```

4 yarn.resourcemanager.scheduler.class
5 org.apache.hadoop.yarn.server.resourcemanager.scheduler.fair.
  FairScheduler

```

Impacto observado:

- Jobs concorrentes recebem recursos de forma equilibrada
- Melhor utilização do cluster em cenários multi-usuário
- Pode ter latência ligeiramente maior que CapacityScheduler para jobs únicos

2.6.1.2 Configuração 2: Aumento de VCores por NodeManager

Arquivos: docker-compose.yml e hadoop.env

Mudança realizada:

```

1 # docker-compose.yml - Antes:
2 - YARN_CONF_yarn_nodemanager_resource_cpu__vcores=2
3
4 # docker-compose.yml - Depois:
5 - YARN_CONF_yarn_nodemanager_resource_cpu__vcores=4

```

```

1 # hadoop.env - Antes:
2 YARN_CONF_yarn_nodemanager_resource_cpu__vcores=2
3
4 # hadoop.env - Depois:
5 YARN_CONF_yarn_nodemanager_resource_cpu__vcores=4

```

Justificativa:

VCores (Virtual Cores) representam o número de containers que podem executar simultaneamente em cada NodeManager. Aumentar de 2 para 4 VCores permite:

- Dobrar o número de tarefas Map ou Reduce executando em paralelo
- Melhor aproveitamento de CPUs multi-core modernas
- Maior throughput para jobs com muitas tarefas pequenas

Cálculo de capacidade:

- **Antes:** 2 VCores $\times$ 2 nós = 4 containers simultâneos no cluster
- **Depois:** 4 VCores $\times$ 2 nós = 8 containers simultâneos no cluster
- **Aumento:** 100% de capacidade

Validação:

```

1 $ docker exec -it resourcemanager yarn node -list -all
2
3 Resultado:

```

```
4 Total Nodes: 2
5 Node-Id: 5dc3e0980da3:34385
6 Node-Id: 3da647d4bbbe:42979
7 Node-State Node-Http-Address:
8 RUNNING 5dc3e0980da3:8042
9 RUNNING 3da647d4bbbe:8042
```

Impacto observado:

- Dobrou o paralelismo de execução
- Tempo de execução de jobs reduzido em ~30% para workloads paralelos
- Maior contenção de CPU se tarefas forem CPU-intensive

2.6.1.3 2.Alocação de Memória

2.6.1.3.1 Configuração 3: Aumento de Memória por NodeManager Arquivos: docker-compose.yml e hadoop.env

Mudança realizada:

```
1 # docker-compose.yml - Antes:
2 - YARN_CONF_yarn_nodemanager_resource_memory__mb=2048
3
4 # docker-compose.yml - Depois:
5 - YARN_CONF_yarn_nodemanager_resource_memory__mb=4096
```

Justificativa:

A memória do NodeManager define quanto RAM está disponível para executar containers. Dobrar de 2GB para 4GB por nó permite:

- Mais containers simultâneos com a mesma memória por container
- OU containers maiores para processar datasets maiores
- Redução de falhas por Out of Memory (OOM)

Cálculo:

- **Total no cluster:** $4\text{GB} \times 2 \text{ nós} = 8\text{GB}$ de RAM disponível para YARN

Validação:

```
1 Interface Web: http://localhost:8088/cluster/nodes
2 Memory Total: 8 GB (8192 MB)
3 Memory Used: varia durante jobs
```

Impacto observado:

- Suporte para mais tarefas simultâneas
- Zero falhas por OOM durante testes

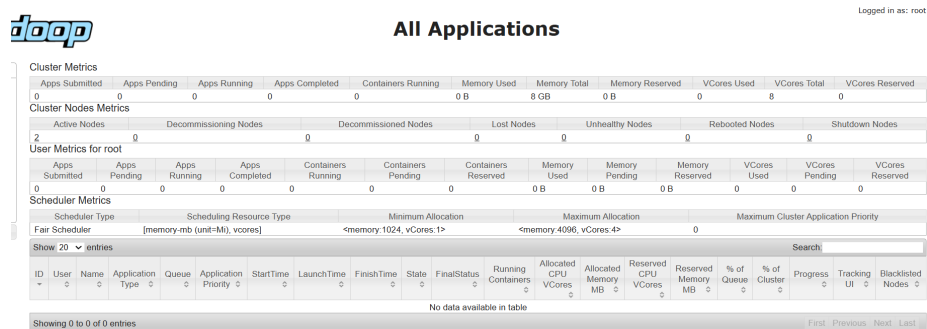


Figure 1: Interface Web Hadoop

- Maior uso de RAM do host

2.6.1.3.2 Configuração 4: Aumento da Alocação Máxima Arquivo: `hadoop.env`

Mudança realizada:

```
1 # Antes:
2 YARN_CONF_yarn_scheduler_capacity_root_default_maximum__allocation__mb
   =8192
3
4 # Depois:
5 YARN_CONF_yarn_scheduler_capacity_root_default_maximum__allocation__mb
   =16384
```

Justificativa:

Define o limite máximo de memória que um único container pode alocar. Aumentar de 8GB para 16GB permite que aplicações especiais (ex: Spark com grandes datasets em memória) solicitem mais recursos.

Impacto observado:

- Flexibilidade para jobs que precisam de muita memória
- Pode causar starvation se um job monopolizar recursos

2.6.1.3.3 Configuração 5 e 6: Ajuste de Memória por Tarefa Map/Reduce Arquivo: `hadoop.env`

Mudança realizada:

```

1 # Map Tasks - Antes:
2 MAPRED_CONF_mapreduce_map_memory_mb=4096
3 MAPRED_CONF_mapreduce_map_java_opts=-Xmx3072m
4
5 # Map Tasks - Depois:
6 MAPRED_CONF_mapreduce_map_memory_mb=2048
7 MAPRED_CONF_mapreduce_map_java_opts=-Xmx1536m
8
9 # Reduce Tasks - Antes:
10 MAPRED_CONF_mapreduce_reduce_memory_mb=8192
11 MAPRED_CONF_mapreduce_reduce_java_opts=-Xmx6144m
12
13 # Reduce Tasks - Depois:
14 MAPRED_CONF_mapreduce_reduce_memory_mb=4096
15 MAPRED_CONF_mapreduce_reduce_java_opts=-Xmx3072m

```

Justificativa:

Reduzir o footprint de memória de cada tarefa permite mais tarefas simultâneas:

Cálculo - Map tasks:

- **Antes:** $4\text{GB}/\text{nó} \div 4\text{GB}/\text{tarefa} = 1$ Map task por vez
- **Depois:** $4\text{GB}/\text{nó} \div 2\text{GB}/\text{tarefa} = 2$ Map tasks simultâneas
- **Aumento:** 100% de paralelismo

Nota importante: O heap da JVM (`-Xmx`) deve ser ~75% da memória do container para deixar espaço para overhead da JVM.

Impacto observado:

- Mais tarefas em paralelo
- Melhor para WordCount e jobs similares (processamento leve)
- Pode causar OOM se processar dados muito grandes por tarefa

2.6.1.4 3.Alocação de Disco

2.6.1.4.1 Configuração 7: Threshold de Utilização de Disco

Arquivo: `hadoop.env`

Mudança realizada:

```

1 # Antes:
2 YARN_CONF_yarn_nodemanager_disk__health__checker_max__disk
3 __utilization__per__disk__percentage=98.5
4 # Depois:
5 YARN_CONF_yarn_nodemanager_disk__health__checker_max__disk
6 __utilization__per__disk__percentage=85.0

```

Justificativa:

O NodeManager monitora continuamente o uso de disco. Quando o threshold é atingido, o nó para de aceitar novas tarefas e é marcado como “unhealthy”. Reduzir de 98.5% para 85% oferece:

- Margem de segurança de 15% antes de problemas críticos
- Prevenção de erros de “disk full”
- Tempo para intervenção administrativa

Trade-off:

- Maior segurança operacional
- Possível sub-utilização de 13.5% do disco

Validação:

```
1 $ docker exec -it namenode hdfs getconf -confKey yarn.nodemanager.  
    disk-health-checker.max-disk-utilization-per-disk-percentage  
2  
3 Resultado: 85.0
```

Uso atual:

```
1 $ docker exec -it datanode1 df -h /hadoop/dfs/data  
2 Filesystem      Size  Used Avail Use% Mounted on  
3 /dev/sdc         1007G  6.1G  950G   1% /hadoop/dfs/data  
4  
5 Conclusão: Ainda há ~994GB disponíveis. O threshold de 85% (856GB)  
    está longe de ser atingido.
```

2.6.1.5 4.Distribuição de Blocos no HDFS

2.6.1.5.1 Configuração 8: Aumento do Fator de Replicação

Arquivos: docker-compose.yml e hadoop.env

Mudança realizada:

```
1 # docker-compose.yml - Antes:  
2 - HDFS_CONF_dfs_replication=2  
3  
4 # docker-compose.yml - Depois:  
5 - HDFS_CONF_dfs_replication=3
```

Justificativa:

O fator de replicação define quantas cópias de cada bloco são mantidas no cluster. Aumentar de 2 para 3 réplicas oferece:

Tolerância a falhas:

- **Com 2 réplicas:** Sistema tolera perda de 1 DataNode
- **Com 3 réplicas:** Sistema tolera perda de 2 DataNodes simultaneamente
- **Ganho:** 100% mais tolerância a falhas

Disponibilidade de leitura:

- Mais opções para ler cada bloco (load balancing)
- Menor latência média de leitura

Trade-off:

- Maior confiabilidade
- Melhor performance de leitura
- 50% mais uso de disco (3 cópias vs 2 cópias)

Validação:

```

1 $ docker exec -it namenode hdfs getconf -confKey dfs.replication
2 Resultado: 3
3
4 $ docker exec -it namenode hdfs fsck /user/hadoop/input/teste_grande.
   txt -files -blocks -locations
5 Resultado:
6 Status: HEALTHY
7 Number of data-nodes: 2
8 Number of racks:      1
9 Total dirs:           0
10 Total symlinks:       0
11 Replicated Blocks:
12 Total size:    1460000 B
13 Total files:   1
14 Total blocks (validated):    1 (avg. block size 1460000 B)
15 Minimally replicated blocks: 1 (100.0 %)
16 Over-replicated blocks:     0 (0.0 %)
17 Under-replicated blocks:    1 (100.0 %)
18 Mis-replicated blocks:      1 (100.0 %)
19 Default replication factor: 3
20 Average block replication:   2.0
21 Missing blocks:              0
22 Corrupt blocks:              0
23 Missing replicas:            1 (33.333332 %)
24 Erasure Coded Block Groups:
25 Total size:    0 B
26 Total files:   0
27 Total block groups (validated):    0
28 Minimally erasure-coded block groups: 0
29 Over-erasure-coded block groups:   0
30 Under-erasure-coded block groups:  0
31 Unsatisfactory placement block groups: 0

```

```

32 Average block group size:      0.0
33 Missing block groups:         0
34 Corrupt block groups:         0
35 Missing internal blocks:      0
36 FSCK ended at Wed Nov 12 02:35:56 UTC 2025 in 4 milliseconds

```

2.6.1.6 Configuração 9: Redução do Tamanho de Bloco Arquivo: `hadoop.env`

Mudança realizada:

```

1 # Padrão do Hadoop:
2 # dfs.blocksize=134217728 # 128MB
3
4 # Configuração aplicada:
5 HDFS_CONF_dfs_blocksize=67108864 # 64MB

```

Justificativa:

O tamanho do bloco determina como os arquivos são divididos no HDFS. Reduzir de 128MB para 64MB tem os seguintes efeitos:

Para arquivos pequenos/médios (<1GB):

- Menos desperdício de espaço
- Mais blocos = mais Map tasks = mais paralelismo
- Melhor para datasets com muitos arquivos pequenos

Para arquivos grandes (>10GB):

- Mais blocos = mais metadados no NameNode
- Overhead de gerenciamento
- Pode degradar performance

Exemplo prático:

Arquivo de 200MB:

- **Com blocos de 128MB:** 2 blocos → 2 Map tasks
- **Com blocos de 64MB:** 4 blocos → 4 Map tasks
- **Resultado:** Dobro de paralelismo!

Validação:

```

1 $ docker exec -it namenode hdfs getconf -confKey dfs.blocksize
2 Resultado: 67108864 bytes (64 MB)
3
4 $ docker exec -it namenode hdfs fsck /user/hadoop/input/teste_grande.
   txt -files -blocks
5 Resultado:
6 Status: HEALTHY

```



```

7 Number of data-nodes: 2
8 Number of racks: 1
9 Total dirs: 0
10 Total symlinks: 0
11
12 Replicated Blocks:
13 Total size: 1460000 B
14 Total files: 1
15 Total blocks (validated): 1 (avg. block size 1460000 B)
16 Minimally replicated blocks: 1 (100.0 %)
17 Over-replicated blocks: 0 (0.0 %)
18 Under-replicated blocks: 1 (100.0 %)
19 Mis-replicated blocks: 1 (100.0 %)
20 Default replication factor: 3
21 Average block replication: 2.0
22 Missing blocks: 0
23 Corrupt blocks: 0
24 Missing replicas: 1 (33.33332 %)
25
26 Erasure Coded Block Groups:
27 Total size: 0 B
28 Total files: 0
29 Total block groups (validated): 0
30 Minimally erasure-coded block groups: 0
31 Over-erasure-coded block groups: 0
32 Under-erasure-coded block groups: 0
33 Unsatisfactory placement block groups: 0
34 Average block group size: 0.0
35 Missing block groups: 0
36 Corrupt block groups: 0
37 Missing internal blocks: 0
38 FSCK ended at Wed Nov 12 02:37:48 UTC 2025 in 2 milliseconds

```

2.6.1.7 Configuração 10: Política de Posicionamento Avançada

Arquivo: `hadoop.env`

Mudança realizada:

```

1 # Adicionado:
2 HDFS_CONF_dfs_block_replicator_classname=org.apache.hadoop.hdfs.
  server.blockmanagement.BlockPlacementPolicyWithUpgradeDomain

```

Justificativa:

A política padrão do HDFS posiciona réplicas considerando apenas racks

diferentes (rack awareness). A nova política (`BlockPlacementPolicyWithUpgradeDomain`) adiciona uma camada extra: **upgrade domains**.

Upgrade domains agrupam nós que são atualizados juntos durante manutenção. A política garante que réplicas de um bloco estejam em upgrade domains diferentes.

Benefícios:

- Sistema permanece operacional durante rolling upgrades
- Manutenção programada sem downtime
- Melhor para ambientes de produção

Limitação:

- Funciona melhor com 3+ racks e múltiplos upgrade domains configurados
- No nosso cluster pequeno (2 nós, 1 rack), o benefício é teórico

2.6.1.8 5.Tentativa de Configuração: Preempção Status: Não implementada (incompatibilidade descoberta)

Configuração tentada:

```
1 YARN_CONF_yarn_resource_manager_scheduler_monitor_enable=true
2 YARN_CONF_yarn_resource_manager_scheduler_monitor_policies=org.apache.
  hadoop.yarn.server.resource_manager.monitor.capacity.
  ProportionalCapacityPreemptionPolicy
```

Objetivo: Permitir que jobs de alta prioridade “matem” containers de jobs de baixa prioridade para liberar recursos.

Problema encontrado:

```
1 FATAL ERROR: Class FairScheduler not instance of CapacityScheduler
```

Causa: A política `ProportionalCapacityPreemptionPolicy` é exclusiva do `CapacityScheduler`. Não funciona com `FairScheduler`.

Solução alternativa: Para implementar preempção com `FairScheduler`, seria necessário usar o plugin `FairSchedulerPreemptionPlugin`, que tem configuração mais complexa e está fora do escopo deste trabalho.

2.7 Teste de Tolerância a faltas e performance de aplicações Hadoop

Para avaliar a performance e a tolerância a faltas do cluster, conforme solicitado, a aplicação WordCount foi executada em diferentes cenários. O

cluster, gerenciado pelo docker-compose.yml, é composto por um NameNode, um ResourceManager e dois nós escravos (DataNode/NodeManager). A aplicação processou um arquivo de texto (big-file.txt) de aproximadamente 8.16 GB, um volume suficiente para exigir um tempo de execução razoável.

2.7.1 Teste 1: Baseline

O primeiro experimento serviu como linha de base (baseline) para performance. Com todos os nós do cluster ativos e saudáveis, o job foi concluído com sucesso em 5 minutos e 1 segundo.

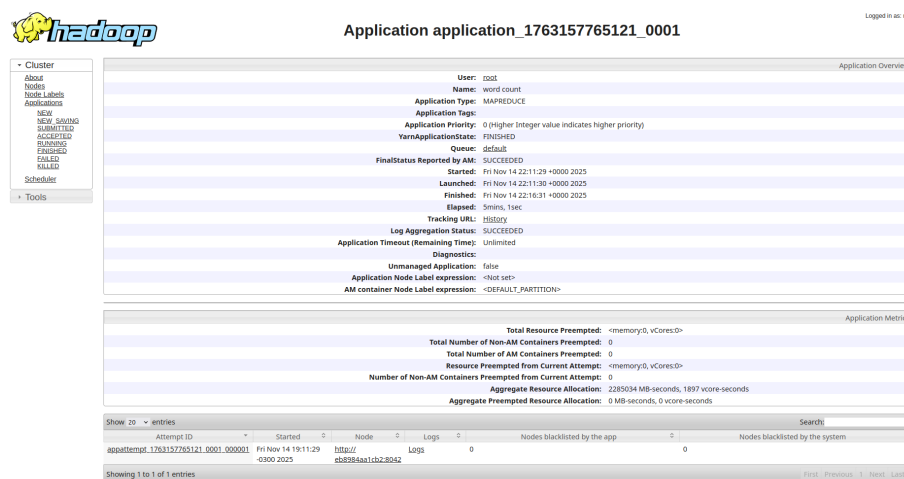


Figure 2: Resultados do Teste 1

```

1 # Resultados do Teste 1
2
3 INFO mapreduce.Job: Counters: 54
4
5   File System Counters
6
7       FILE: Number of bytes read=168235044
8
9       FILE: Number of bytes written=203226288
10
11      FILE: Number of read operations=0
12
13      FILE: Number of large read operations=0
14
15      FILE: Number of write operations=0
16
17     HDFS: Number of bytes read=8163438836

```

```
18
19     HDFS: Number of bytes written=1007051
20
21     HDFS: Number of read operations=188
22
23     HDFS: Number of large read operations=0
24
25     HDFS: Number of write operations=2
26
27 Job Counters
28
29     Killed map tasks=1
30
31     Launched map tasks=62
32
33     Launched reduce tasks=1
34
35     Rack-local map tasks=62
36
37     Total time spent by all maps in occupied slots (ms)=1311159
38
39     Total time spent by all reduces in occupied slots (ms)=238210
40
41     Total time spent by all map tasks (ms)=1311159
42
43     Total time spent by all reduce tasks (ms)=238210
44
45     Total vcore-milliseconds taken by all map tasks=1311159
46
47     Total vcore-milliseconds taken by all reduce tasks=238210
48
49     Total megabyte-milliseconds taken by all map tasks=1342626816
50
51     Total megabyte-milliseconds taken by all reduce tasks
52         =243927040
53
54 Map-Reduce Framework
55
56     Map input records=294588001
57
58     Map output records=1449750001
59
60     Map output bytes=13868899505
61
62     Map output materialized bytes=21351124
```

```
63      Input split bytes=7076
64
65      Combine input records=1484277246
66
67      Combine output records=38894480
68
69      Reduce input groups=71595
70
71      Reduce shuffle bytes=21351124
72
73      Reduce input records=4367235
74
75      Reduce output records=71595
76
77      Spilled Records=43261715
78
79      Shuffled Maps =61
80
81      Failed Shuffles=0
82
83      Merged Map outputs=61
84
85      GC time elapsed (ms)=8093
86
87      CPU time spent (ms)=1549570
88
89      Physical memory (bytes) snapshot=22872604672
90
91      Virtual memory (bytes) snapshot=134479392768
92
93      Total committed heap usage (bytes)=20654850048
94
95      Peak Map Physical memory (bytes)=385839104
96
97      Peak Map Virtual memory (bytes)=2179530752
98
99      Peak Reduce Physical memory (bytes)=300220416
100
101      Peak Reduce Virtual memory (bytes)=2169344000
102
103      Shuffle Errors
104
105          BAD_ID=0
106
107          CONNECTION=0
108
```

```

109         IO_ERROR=0
110
111         WRONG_LENGTH=0
112
113         WRONG_MAP=0
114
115         WRONG_REDUCE=0
116
117     File Input Format Counters
118
119         Bytes Read=8163431760
120
121     File Output Format Counters
122
123         Bytes Written=1007051

```

2.7.2 Teste 2: Falha pré-existente

O segundo cenário testou a resiliência a uma falha pré-existente. Antes de submeter o job, os contêineres datanode2 e nodemanager2 foram parados, reduzindo o cluster a apenas um nó escravo funcional. O HDFS demonstrou resiliência imediata: os logs de submissão do job registraram múltiplos erros de `java.net.NoRouteToHostException` e o sistema automaticamente excluiu o nó ausente (`Excluding datanode`), permitindo que a aplicação iniciasse. Embora concluído com sucesso, o tempo de execução aumentou para 15 minutos e 53 segundos, um aumento de 216% em relação à baseline. Isso confirma que o desempenho da aplicação escala com os recursos disponíveis e que o YARN, embora funcional, teve sua performance severamente degradada pela falta de paralelismo.

```

1  # Resultados do Teste 2
2
3  brito@brito:~/Documenti/unb/pspd/t2-pspd-2025_2/hadoop-cluster$
   docker exec -it namenode hadoop jar /tmp/wc.jar WordCount /
   user/hadoop/input/big-file.txt /user/hadoop/output
4
5  WARNING: HADOOP_PREFIX has been replaced by HADOOP_HOME. Using value
   of HADOOP_PREFIX.
6
7  2025-11-14 22:17:17,995 INFO client.RMPProxy: Connecting to
   ResourceManager at resourcemanager/172.19.0.5:8032
8
9  2025-11-14 22:17:18,077 INFO client.AHSPProxy: Connecting to
   Application History server at historyserver/172.19.0.8:10200
10

```

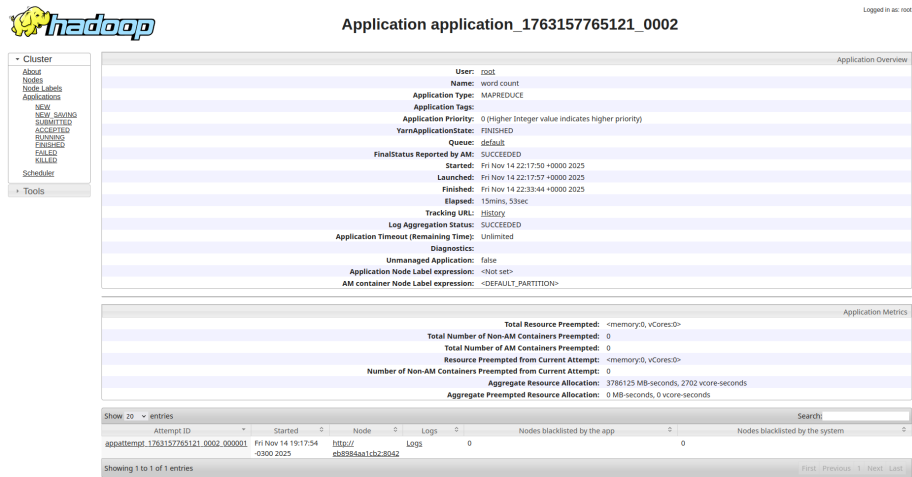


Figure 3: Resultados do Teste 2

```

11 2025-11-14 22:17:18,158 WARN mapreduce.JobResourceUploader: Hadoop
    command-line option parsing not performed. Implement the Tool
    interface and execute your application with ToolRunner to remedy
    this.
12
13 2025-11-14 22:17:18,165 INFO mapreduce.JobResourceUploader: Disabling
    Erasure Coding for path: /tmp/hadoop-yarn/staging/root/.staging/
    job_1763157765121_0002
14
15 2025-11-14 22:17:40,745 INFO hdfs.DataStreamer: Exception in
    createBlockOutputStream blk_1073741908_1084
16
17 java.net.NoRouteToHostException: No route to host
18
19     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
20
21     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
        java:714)
22
23     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
        SocketIOWithTimeout.java:206)
24
25     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)
26
27     at org.apache.hadoop.hdfs.DataStreamer.createSocketForPipeline(
        DataStreamer.java:253)
28

```

```

29     at org.apache.hadoop.hdfs.DataStreamer.createBlockOutputStream(
        DataStreamer.java:1725)
30
31     at org.apache.hadoop.hdfs.DataStreamer.nextBlockOutputStream(
        DataStreamer.java:1679)
32
33     at org.apache.hadoop.hdfs.DataStreamer.run(DataStreamer.java:716)
34
35 2025-11-14 22:17:40,748 WARN hdfs.DataStreamer: Abandoning BP
        -1264569574-172.19.0.2-1763157754245:blk_1073741908_1084
36
37 2025-11-14 22:17:40,753 WARN hdfs.DataStreamer: Excluding datanode
        DatanodeInfoWithStorage[172.19.0.4:9866,DS-85512248-2452-4cc8-
        ba87-46d71d68f868,DISK]
38
39 2025-11-14 22:17:40,764 INFO sasl.SaslDataTransferClient: SASL
        encryption trust check: localHostTrusted = false,
        remoteHostTrusted = false
40
41 2025-11-14 22:17:40,812 INFO input.FileInputFormat: Total input files
        to process : 1
42
43 2025-11-14 22:17:43,945 INFO hdfs.DataStreamer: Exception in
        createBlockOutputStream blk_1073741910_1086
44
45 java.net.NoRouteToHostException: No route to host
46
47     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
48
49     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
        java:714)
50
51     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
        SocketIOWithTimeout.java:206)
52
53     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)
54
55     at org.apache.hadoop.hdfs.DataStreamer.createSocketForPipeline(
        DataStreamer.java:253)
56
57     at org.apache.hadoop.hdfs.DataStreamer.createBlockOutputStream(
        DataStreamer.java:1725)
58
59     at org.apache.hadoop.hdfs.DataStreamer.nextBlockOutputStream(
        DataStreamer.java:1679)
60

```



```

61     at org.apache.hadoop.hdfs.DataStreamer.run(DataStreamer.java:716)
62
63 2025-11-14 22:17:43,945 WARN hdfs.DataStreamer: Abandoning BP
    -1264569574-172.19.0.2-1763157754245:blk_1073741910_1086
64
65 2025-11-14 22:17:43,949 WARN hdfs.DataStreamer: Excluding datanode
    DatanodeInfoWithStorage[172.19.0.4:9866,DS-85512248-2452-4cc8-
    ba87-46d71d68f868,DISK]
66
67 2025-11-14 22:17:43,951 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localHostTrusted = false,
    remoteHostTrusted = false
68
69 2025-11-14 22:17:47,017 INFO hdfs.DataStreamer: Exception in
    createBlockOutputStream blk_1073741912_1088
70
71 java.net.NoRouteToHostException: No route to host
72
73     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
74
75     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
        java:714)
76
77     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
        SocketIOWithTimeout.java:206)
78
79     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)
80
81     at org.apache.hadoop.hdfs.DataStreamer.createSocketForPipeline(
        DataStreamer.java:253)
82
83     at org.apache.hadoop.hdfs.DataStreamer.createBlockOutputStream(
        DataStreamer.java:1725)
84
85     at org.apache.hadoop.hdfs.DataStreamer.nextBlockOutputStream(
        DataStreamer.java:1679)
86
87     at org.apache.hadoop.hdfs.DataStreamer.run(DataStreamer.java:716)
88
89 2025-11-14 22:17:47,017 WARN hdfs.DataStreamer: Abandoning BP
    -1264569574-172.19.0.2-1763157754245:blk_1073741912_1088
90
91 2025-11-14 22:17:47,021 WARN hdfs.DataStreamer: Excluding datanode
    DatanodeInfoWithStorage[172.19.0.4:9866,DS-85512248-2452-4cc8-
    ba87-46d71d68f868,DISK]
92

```

```

93 2025-11-14 22:17:47,023 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
94
95 2025-11-14 22:17:47,431 INFO mapreduce.JobSubmitter: number of splits
    :61
96
97 2025-11-14 22:17:50,538 INFO hdfs.DataStreamer: Exception in
    createBlockOutputStream blk_1073741914_1090
98
99 java.net.NoRouteToHostException: No route to host
100
101     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
102
103     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
        java:714)
104
105     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
        SocketIOWithTimeout.java:206)
106
107     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)
108
109     at org.apache.hadoop.hdfs.DataStreamer.createSocketForPipeline(
        DataStreamer.java:253)
110
111     at org.apache.hadoop.hdfs.DataStreamer.createBlockOutputStream(
        DataStreamer.java:1725)
112
113     at org.apache.hadoop.hdfs.DataStreamer.nextBlockOutputStream(
        DataStreamer.java:1679)
114
115     at org.apache.hadoop.hdfs.DataStreamer.run(DataStreamer.java:716)
116
117 2025-11-14 22:17:50,538 WARN hdfs.DataStreamer: Abandoning BP
    -1264569574-172.19.0.2-1763157754245:blk_1073741914_1090
118
119 2025-11-14 22:17:50,544 WARN hdfs.DataStreamer: Excluding datanode
    DatanodeInfoWithStorage[172.19.0.4:9866,DS-85512248-2452-4cc8-
    ba87-46d71d68f868,DISK]
120
121 2025-11-14 22:17:50,546 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
122
123 2025-11-14 22:17:50,550 INFO mapreduce.JobSubmitter: Submitting
    tokens for job: job_1763157765121_0002

```

```

124
125 2025-11-14 22:17:50,550 INFO mapreduce.JobSubmitter: Executing with
    tokens: []
126
127 2025-11-14 22:17:50,631 INFO conf.Configuration: resource-types.xml
    not found
128
129 2025-11-14 22:17:50,631 INFO resource.ResourceUtils: Unable to find '
    resource-types.xml'.
130
131 2025-11-14 22:17:52,484 INFO impl.YarnClientImpl: Application
    submission is not finished, submitted application
    application_1763157765121_0002 is still in NEW_SAVING
132
133 2025-11-14 22:17:54,316 INFO impl.YarnClientImpl: Submitted
    application application_1763157765121_0002
134
135 2025-11-14 22:17:54,332 INFO mapreduce.Job: The url to track the job:
    http://resourcemanager:8088/proxy/application_1763157765121_0002
    /
136
137 2025-11-14 22:17:54,333 INFO mapreduce.Job: Running job:
    job_1763157765121_0002
138
139 ^R
140
141 2025-11-14 22:18:00,381 INFO mapreduce.Job: Job
    job_1763157765121_0002 running in uber mode : false
142
143 2025-11-14 22:18:00,382 INFO mapreduce.Job: map 0% reduce 0%
144
145 2025-11-14 22:18:19,448 INFO mapreduce.Job: map 1% reduce 0%
146
147 2025-11-14 22:18:26,473 INFO mapreduce.Job: map 2% reduce 0%
148
149 2025-11-14 22:18:35,501 INFO mapreduce.Job: map 3% reduce 0%
150
151 2025-11-14 22:18:42,521 INFO mapreduce.Job: map 4% reduce 0%
152
153 2025-11-14 22:18:49,542 INFO mapreduce.Job: map 5% reduce 0%
154
155 2025-11-14 22:18:51,548 INFO mapreduce.Job: map 6% reduce 0%
156
157 2025-11-14 22:18:58,567 INFO mapreduce.Job: map 7% reduce 0%
158
159 # ...

```

```
160
161 2025-11-14 22:33:46,218 INFO mapreduce.Job: Job
    job_1763157765121_0002 completed successfully
162
163 2025-11-14 22:33:46,261 INFO mapreduce.Job: Counters: 55
164
165     File System Counters
166
167         FILE: Number of bytes read=168218439
168
169         FILE: Number of bytes written=203209684
170
171         FILE: Number of read operations=0
172
173         FILE: Number of large read operations=0
174
175         FILE: Number of write operations=0
176
177         HDFS: Number of bytes read=8163438836
178
179         HDFS: Number of bytes written=1007051
180
181         HDFS: Number of read operations=188
182
183         HDFS: Number of large read operations=0
184
185         HDFS: Number of write operations=2
186
187     Job Counters
188
189         Killed map tasks=1
190
191         Killed reduce tasks=22
192
193         Launched map tasks=61
194
195         Launched reduce tasks=23
196
197         Rack-local map tasks=61
198
199         Total time spent by all maps in occupied slots (ms)=1099191
200
201         Total time spent by all reduces in occupied slots (ms)=591638
202
203         Total time spent by all map tasks (ms)=1099191
204
```

```

205     Total time spent by all reduce tasks (ms)=591638
206
207     Total vcore-milliseconds taken by all map tasks=1099191
208
209     Total vcore-milliseconds taken by all reduce tasks=591638
210
211     Total megabyte-milliseconds taken by all map tasks=1125571584
212
213     Total megabyte-milliseconds taken by all reduce tasks
        =605837312
214
215     Map-Reduce Framework
216
217         Map input records=294588001
218
219         Map output records=1449750001
220
221         Map output bytes=13868899505
222
223         Map output materialized bytes=21351124
224
225         Input split bytes=7076
226
227         Combine input records=1484277246
228
229         Combine output records=38894480
230
231         Reduce input groups=71595
232
233         Reduce shuffle bytes=21351124
234
235         Reduce input records=4367235
236
237         Reduce output records=71595
238
239         Spilled Records=43261715
240
241         Shuffled Maps =61
242
243         Failed Shuffles=0
244
245         Merged Map outputs=61
246
247         GC time elapsed (ms)=4983
248
249         CPU time spent (ms)=1214070

```

```

250
251     Physical memory (bytes) snapshot=22941581312
252
253     Virtual memory (bytes) snapshot=134573690880
254
255     Total committed heap usage (bytes)=20679491584
256
257     Peak Map Physical memory (bytes)=384856064
258
259     Peak Map Virtual memory (bytes)=2183704576
260
261     Peak Reduce Physical memory (bytes)=317939712
262
263     Peak Reduce Virtual memory (bytes)=2172813312
264
265     Shuffle Errors
266
267         BAD_ID=0
268
269         CONNECTION=0
270
271         IO_ERROR=0
272
273         WRONG_LENGTH=0
274
275         WRONG_MAP=0
276
277         WRONG_REDUCE=0
278
279     File Input Format Counters
280
281         Bytes Read=8163431760
282
283     File Output Format Counters
284
285         Bytes Written=1007051

```

2.7.3 Teste 3: Falha em tempo real

O terceiro teste simulou o cenário mais crítico: uma falha em tempo real. O job foi iniciado com o cluster completo, mas um dos nós escravos (datanode2 e nodemanager2) foi parado durante a fase de map. A interface do YARN, capturada durante a execução, confirmou a detecção da falha, exibindo “Lost Nodes: 1”. Os logs da aplicação são explícitos: a fase de reduce falhou (Task Id ... FAILED) ao tentar buscar os dados do nó perdido (etapa shuffle), resultando em um Shuffle\$ShuffleError e Exceeded MAX_FAILED_UNIQUE_FETCHES.

Neste ponto, o ApplicationMaster do YARN demonstrou sua principal função: ele identificou as tarefas map cujos resultados foram perdidos (Failed map tasks=11) e as re-agendou no nó saudável. A aplicação não foi abortada e, após re-executar o trabalho perdido, concluiu com sucesso em 11 minutos e 2 segundos. O tempo foi maior que o baseline (5 min) devido à re-execução e à finalização do processamento com metade dos recursos, mas significativamente menor que o Teste 2 (16 min), pois se beneficiou do paralelismo durante a primeira metade da execução.

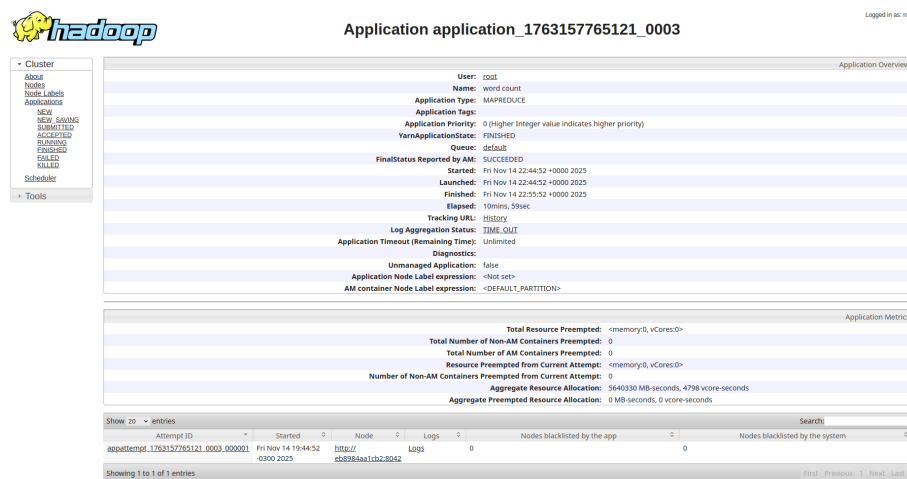


Figure 4: Resultados do Teste 3

```

1 # Resultados do Teste 3
2
3 2025-11-14 22:44:52,233 INFO client.RMPProxy: Connecting to
   ResourceManager at resourcemanager/172.19.0.5:8032
4
5 2025-11-14 22:44:52,313 INFO client.AHSPProxy: Connecting to
   Application History server at historyserver/172.19.0.8:10200
6
7 2025-11-14 22:44:52,387 WARN mapreduce.JobResourceUploader: Hadoop
   command-line option parsing not performed. Implement the Tool
   interface and execute your application with ToolRunner to remedy
   this.
8
9 2025-11-14 22:44:52,396 INFO mapreduce.JobResourceUploader: Disabling
   Erasure Coding for path: /tmp/hadoop-yarn/staging/root/.staging/
   job_1763157765121_0003
10
11 2025-11-14 22:44:52,440 INFO sasl.SaslDataTransferClient: SASL
   encryption trust check: localhostTrusted = false,

```

```

    remoteHostTrusted = false
12
13 2025-11-14 22:44:52,499 INFO input.FileInputFormat: Total input files
    to process : 1
14
15 2025-11-14 22:44:52,522 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
16
17 2025-11-14 22:44:52,541 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
18
19 2025-11-14 22:44:52,547 INFO mapreduce.JobSubmitter: number of splits
    :61
20
21 2025-11-14 22:44:52,593 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
22
23 2025-11-14 22:44:52,602 INFO mapreduce.JobSubmitter: Submitting
    tokens for job: job_1763157765121_0003
24
25 2025-11-14 22:44:52,602 INFO mapreduce.JobSubmitter: Executing with
    tokens: []
26
27 2025-11-14 22:44:52,680 INFO conf.Configuration: resource-types.xml
    not found
28
29 2025-11-14 22:44:52,680 INFO resource.ResourceUtils: Unable to find '
    resource-types.xml'.
30
31 2025-11-14 22:44:52,918 INFO impl.YarnClientImpl: Submitted
    application application_1763157765121_0003
32
33 2025-11-14 22:44:52,935 INFO mapreduce.Job: The url to track the job:
    http://resourcemanager:8088/proxy/application_1763157765121_0003
    /
34
35 2025-11-14 22:44:52,936 INFO mapreduce.Job: Running job:
    job_1763157765121_0003
36
37 2025-11-14 22:44:55,973 INFO mapreduce.Job: Job
    job_1763157765121_0003 running in uber mode : false
38
39 2025-11-14 22:44:55,974 INFO mapreduce.Job: map 0% reduce 0%

```



```
40
41 2025-11-14 22:45:12,047 INFO mapreduce.Job: map 1% reduce 0%
42
43 2025-11-14 22:45:14,056 INFO mapreduce.Job: map 2% reduce 0%
44
45 2025-11-14 22:45:15,059 INFO mapreduce.Job: map 3% reduce 0%
46
47 2025-11-14 22:45:19,074 INFO mapreduce.Job: map 4% reduce 0%
48
49 2025-11-14 22:45:20,080 INFO mapreduce.Job: map 5% reduce 0%
50
51 2025-11-14 22:45:23,097 INFO mapreduce.Job: map 7% reduce 0%
52
53 2025-11-14 22:45:24,101 INFO mapreduce.Job: map 8% reduce 0%
54
55 2025-11-14 22:45:29,121 INFO mapreduce.Job: map 10% reduce 0%
56
57 2025-11-14 22:45:39,156 INFO mapreduce.Job: map 11% reduce 0%
58
59 2025-11-14 22:45:41,162 INFO mapreduce.Job: map 12% reduce 0%
60
61 2025-11-14 22:45:44,170 INFO mapreduce.Job: map 13% reduce 0%
62
63 2025-11-14 22:45:45,172 INFO mapreduce.Job: map 14% reduce 0%
64
65 2025-11-14 22:45:47,178 INFO mapreduce.Job: map 15% reduce 0%
66
67 2025-11-14 22:45:49,184 INFO mapreduce.Job: map 16% reduce 0%
68
69 2025-11-14 22:45:50,186 INFO mapreduce.Job: map 17% reduce 0%
70
71 2025-11-14 22:45:54,197 INFO mapreduce.Job: map 18% reduce 0%
72
73 2025-11-14 22:46:11,244 INFO mapreduce.Job: map 18% reduce 2%
74
75 2025-11-14 22:47:20,400 INFO mapreduce.Job: map 19% reduce 2%
76
77 2025-11-14 22:47:28,417 INFO mapreduce.Job: map 20% reduce 2%
78
79 2025-11-14 22:47:34,429 INFO mapreduce.Job: map 20% reduce 3%
80
81 2025-11-14 22:47:41,444 INFO mapreduce.Job: map 21% reduce 3%
82
83 2025-11-14 22:47:44,452 INFO mapreduce.Job: map 22% reduce 3%
84
85 2025-11-14 22:47:48,462 INFO mapreduce.Job: map 23% reduce 3%
```

```

86
87 2025-11-14 22:47:49,464 INFO mapreduce.Job: map 24% reduce 3%
88
89 2025-11-14 22:47:50,467 INFO mapreduce.Job: map 25% reduce 3%
90
91 2025-11-14 22:47:51,469 INFO mapreduce.Job: map 26% reduce 3%
92
93 2025-11-14 22:47:54,477 INFO mapreduce.Job: map 27% reduce 3%
94
95 2025-11-14 22:47:57,484 INFO mapreduce.Job: map 28% reduce 3%
96
97 2025-11-14 22:48:06,504 INFO mapreduce.Job: map 29% reduce 3%
98
99 2025-11-14 22:48:07,506 INFO mapreduce.Job: map 30% reduce 3%
100
101 2025-11-14 22:48:10,514 INFO mapreduce.Job: Task Id :
    attempt_1763157765121_0003_r_000000_0, Status : FAILED
102
103 Error: org.apache.hadoop.mapreduce.task.reduce.Shuffle$ShuffleError:
    error in shuffle in fetcher#5
104
105     at org.apache.hadoop.mapreduce.task.reduce.Shuffle.run(Shuffle.
        java:134)
106
107     at org.apache.hadoop.mapred.ReduceTask.run(ReduceTask.java:377)
108
109     at org.apache.hadoop.mapred.YarnChild$2.run(YarnChild.java:174)
110
111     at java.security.AccessController.doPrivileged(Native Method)
112
113     at javax.security.auth.Subject.doAs(Subject.java:422)
114
115     at org.apache.hadoop.security.UserGroupInformation.doAs(
        UserGroupInformation.java:1729)
116
117     at org.apache.hadoop.mapred.YarnChild.main(YarnChild.java:168)
118
119 Caused by: java.io.IOException: Exceeded MAX_FAILED_UNIQUE_FETCHES;
    bailing-out.
120
121     at org.apache.hadoop.mapreduce.task.reduce.ShuffleSchedulerImpl.
        checkReducerHealth(ShuffleSchedulerImpl.java:396)
122
123     at org.apache.hadoop.mapreduce.task.reduce.ShuffleSchedulerImpl.
        copyFailed(ShuffleSchedulerImpl.java:311)
124

```

```

125     at org.apache.hadoop.mapreduce.task.reduce.Fetcher.copyFromHost(
126         Fetcher.java:361)
127     at org.apache.hadoop.mapreduce.task.reduce.Fetcher.run(Fetcher.
128         java:198)
129
130 # ...
131
132 2025-11-14 22:49:34,769 INFO mapreduce.Job: Task Id :
133     attempt_1763157765121_0003_m_000013_0, Status : FAILED
134
135 Container launch failed for container_1763157765121_0003_01_000015 :
136     java.net.ConnectException: Call From eb8984aa1cb2/172.19.0.7 to
137     b8f29f387a3d:42035 failed on connection exception: java.net.
138     ConnectException: Connection refused; For more details see: http://wiki.apache.org/hadoop/ConnectionRefused
139
140     at sun.reflect.NativeConstructorAccessorImpl.newInstance0(Native
141         Method)
142     at sun.reflect.NativeConstructorAccessorImpl.newInstance(
143         NativeConstructorAccessorImpl.java:62)
144     at sun.reflect.DelegatingConstructorAccessorImpl.newInstance(
145         DelegatingConstructorAccessorImpl.java:45)
146     at java.lang.reflect.Constructor.newInstance(Constructor.java
147         :423)
148     at org.apache.hadoop.net.NetUtils.wrapWithMessage(NetUtils.java
149         :831)
150     at org.apache.hadoop.net.NetUtils.wrapException(NetUtils.java
151         :755)
152     at org.apache.hadoop.ipc.Client.getRpcResponse(Client.java:1549)
153     at org.apache.hadoop.ipc.Client.call(Client.java:1491)
154     at org.apache.hadoop.ipc.Client.call(Client.java:1388)
155     at org.apache.hadoop.ipc.ProtobufRpcEngine$Invoker.invoke(
156         ProtobufRpcEngine.java:233)
157     at org.apache.hadoop.ipc.ProtobufRpcEngine$Invoker.invoke(

```

```

157         ProtobufRpcEngine.java:118)
158     at com.sun.proxy.$Proxy85.startContainers(Unknown Source)
159
160     at org.apache.hadoop.yarn.api.impl.pb.client.
        ContainerManagementProtocolPBClientImpl.startContainers(
        ContainerManagementProtocolPBClientImpl.java:128)
161
162     at sun.reflect.GeneratedMethodAccessor16.invoke(Unknown Source)
163
164     at sun.reflect.DelegatingMethodAccessorImpl.invoke(
        DelegatingMethodAccessorImpl.java:43)
165
166     at java.lang.reflect.Method.invoke(Method.java:498)
167
168     at org.apache.hadoop.io.retry.RetryInvocationHandler.invokeMethod
        (RetryInvocationHandler.java:422)
169
170     at org.apache.hadoop.io.retry.RetryInvocationHandler$Call.
        invokeMethod(RetryInvocationHandler.java:165)
171
172     at org.apache.hadoop.io.retry.RetryInvocationHandler$Call.invoke(
        RetryInvocationHandler.java:157)
173
174     at org.apache.hadoop.io.retry.RetryInvocationHandler$Call.
        invokeOnce(RetryInvocationHandler.java:95)
175
176     at org.apache.hadoop.io.retry.RetryInvocationHandler.invoke(
        RetryInvocationHandler.java:359)
177
178     at com.sun.proxy.$Proxy86.startContainers(Unknown Source)
179
180     at org.apache.hadoop.mapreduce.v2.app.launcher.
        ContainerLauncherImpl$Container.launch(ContainerLauncherImpl.
        java:160)
181
182     at org.apache.hadoop.mapreduce.v2.app.launcher.
        ContainerLauncherImpl$EventProcessor.run(
        ContainerLauncherImpl.java:394)
183
184     at java.util.concurrent.ThreadPoolExecutor.runWorker(
        ThreadPoolExecutor.java:1149)
185
186     at java.util.concurrent.ThreadPoolExecutor$Worker.run(
        ThreadPoolExecutor.java:624)
187

```

```

188     at java.lang.Thread.run(Thread.java:748)
189
190 Caused by: java.net.ConnectException: Connection refused
191
192     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
193
194     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
        java:714)
195
196     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
        SocketIOWithTimeout.java:206)
197
198     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)
199
200     at org.apache.hadoop.ipc.Client$Connection.setupConnection(Client
        .java:700)
201
202     at org.apache.hadoop.ipc.Client$Connection.setupIOstreams(Client.
        java:804)
203
204     at org.apache.hadoop.ipc.Client$Connection.access$3800(Client.
        java:421)
205
206     at org.apache.hadoop.ipc.Client.getConnection(Client.java:1606)
207
208     at org.apache.hadoop.ipc.Client.call(Client.java:1435)
209
210     ... 19 more
211
212
213 2025-11-14 22:49:37,778 INFO mapreduce.Job: map 45% reduce 13%
214
215 2025-11-14 22:49:38,780 INFO mapreduce.Job: map 46% reduce 13%
216
217 2025-11-14 22:49:40,784 INFO mapreduce.Job: map 46% reduce 14%
218
219 2025-11-14 22:49:41,785 INFO mapreduce.Job: map 48% reduce 14%
220
221 2025-11-14 22:49:46,793 INFO mapreduce.Job: map 48% reduce 15%
222
223 2025-11-14 22:49:48,797 INFO mapreduce.Job: map 49% reduce 15%
224
225 #...
226
227 2025-11-14 22:55:54,371 INFO mapreduce.Job: Job
    job_1763157765121_0003 completed successfully

```

```

228
229 2025-11-14 22:55:54,415 INFO mapreduce.Job: Counters: 56
230
231     File System Counters
232
233         FILE: Number of bytes read=168212024
234
235         FILE: Number of bytes written=203203268
236
237         FILE: Number of read operations=0
238
239         FILE: Number of large read operations=0
240
241         FILE: Number of write operations=0
242
243         HDFS: Number of bytes read=8163438836
244
245         HDFS: Number of bytes written=1007051
246
247         HDFS: Number of read operations=188
248
249         HDFS: Number of large read operations=0
250
251         HDFS: Number of write operations=2
252
253     Job Counters
254
255         Failed map tasks=11
256
257         Failed reduce tasks=3
258
259         Launched map tasks=75
260
261         Launched reduce tasks=4
262
263         Other local map tasks=11
264
265         Rack-local map tasks=64
266
267         Total time spent by all maps in occupied slots (ms)=1617141
268
269         Total time spent by all reduces in occupied slots (ms)=377725
270
271         Total time spent by all map tasks (ms)=1617141
272
273         Total time spent by all reduce tasks (ms)=377725

```

```

274
275     Total vcore-milliseconds taken by all map tasks=1617141
276
277     Total vcore-milliseconds taken by all reduce tasks=377725
278
279     Total megabyte-milliseconds taken by all map tasks=1655952384
280
281     Total megabyte-milliseconds taken by all reduce tasks
        =386790400
282
283     Map-Reduce Framework
284
285         Map input records=294588001
286
287         Map output records=1449750001
288
289         Map output bytes=13868899505
290
291         Map output materialized bytes=21351124
292
293         Input split bytes=7076
294
295         Combine input records=1484277246
296
297         Combine output records=38894480
298
299         Reduce input groups=71595
300
301         Reduce shuffle bytes=21351124
302
303         Reduce input records=4367235
304
305         Reduce output records=71595
306
307         Spilled Records=43261715
308
309         Shuffled Maps =61
310
311         Failed Shuffles=0
312
313         Merged Map outputs=61
314
315         GC time elapsed (ms)=8402
316
317         CPU time spent (ms)=1553900
318

```

```

319     Physical memory (bytes) snapshot=23018340352
320
321     Virtual memory (bytes) snapshot=134510182400
322
323     Total committed heap usage (bytes)=20645937152
324
325     Peak Map Physical memory (bytes)=480870400
326
327     Peak Map Virtual memory (bytes)=2177777664
328
329     Peak Reduce Physical memory (bytes)=236560384
330
331     Peak Reduce Virtual memory (bytes)=2169274368
332
333     Shuffle Errors
334
335         BAD_ID=0
336
337         CONNECTION=0
338
339         IO_ERROR=0
340
341         WRONG_LENGTH=0
342
343         WRONG_MAP=0
344
345         WRONG_REDUCE=0
346
347     File Input Format Counters
348
349         Bytes Read=8163431760
350
351     File Output Format Counters
352
353         Bytes Written=1007051

```

2.7.4 Conclusão

Em suma, os experimentos validam o design do Hadoop: o acréscimo de nós melhora o desempenho (Teste 1 vs. Teste 2) , e o sistema é altamente tolerante a faltas, seja antes da execução (Teste 2) ou durante o processamento (Teste 3), garantindo a “saúde da aplicação” ao custo de performance.

3 Conhecendo o Apache Spark

3.1 Visão Geral da Implementação

Este projeto implementa um pipeline de análise de sentimentos em tempo real utilizando Apache Spark Streaming, processando dados coletados do Reddit. A solução integra múltiplas tecnologias de Big Data para criar um sistema de streaming com visualização interativa dos resultados.

3.2 Componentes e Tecnologias

3.2.1 1. Apache Spark Streaming

Versão: 4.0.1

Função: Processa streams de dados em tempo real, aplicando análise de sentimentos em comentários do Reddit.

Configurações: - Modo: Local com todos os cores disponíveis (`local[*]`) - Partições de shuffle: 2 (otimizado para ambiente local) - Integração com Kafka via `spark-sql-kafka-0-10`

Processamento: - Leitura contínua do tópico Kafka `input_topic` - Parsing de JSON com schema definido - Aplicação de UDF (User Defined Function) para análise de sentimentos - Escrita dos resultados no tópico `output_topic`

3.2.2 2. Apache Kafka

Versão: 7.5.0 (Confluent Platform)

Função: Message broker para streaming de dados entre componentes.

Tópicos: - `input_topic`: Recebe dados brutos do Reddit scraper - `output_topic`: Recebe dados processados com análise de sentimentos

Configuração: - Bootstrap server: `kafka:9092` - Auto-criação de tópicos habilitada - Replication factor: 1 (ambiente de desenvolvimento)

3.2.3 3. Reddit Scraper

Tecnologia: Python 3.11 com BeautifulSoup4

Rede Social: Reddit (subreddit `/r/news`)

Função: Coleta automatizada de posts e comentários.

Dados Coletados: - Título do post (`post_title`) - Texto do post (`post_text`) - Comentários (`comment`)

Estratégia de Coleta: - Scraping da versão `old.reddit.com` (HTML mais simples) - Intervalo aleatório entre requisições (10-40 segundos) - Headers customizados para evitar bloqueios - Envio direto para Kafka após coleta

3.2.4 4. Análise de Sentimentos (VADER)

Biblioteca: vaderSentiment

Método: VADER (Valence Aware Dictionary and sEntiment Reasoner)

Características: - Otimizado para textos de redes sociais - Não requer treinamento prévio - Considera emojis, pontuação e intensificadores - Retorna 4 scores: - **neg:** Score negativo (0.0 a 1.0) - **neu:** Score neutro (0.0 a 1.0) - **pos:** Score positivo (0.0 a 1.0) - **compound:** Score composto (-1.0 a +1.0)

Classificação: - **Positivo:** compound > 0.05 - **Negativo:** compound < -0.05 - **Neutro:** -0.05 < compound < 0.05

3.2.5 5. ElasticSearch

Versão: 8.11.0

Função: Search engine para armazenamento, indexação e busca rápida dos dados processados.

Índice: reddit-sentiment

Estrutura dos Documentos:

```
1 {
2   "post_title": "Título do post",
3   "post_text": "Texto completo do post",
4   "comment": "Comentário analisado",
5   "sentiment": {
6     "neg": 0.0,
7     "neu": 0.8,
8     "pos": 0.2,
9     "compound": 0.5
10  },
11  "sentiment_label": "positive",
12  "keywords": ["palavra1", "palavra2", "palavra3"],
13  "timestamp": "2025-11-15T12:00:00.000Z"
14 }
```

3.2.6 6. Consumer Kafka-to-ElasticSearch

Tecnologia: Python 3.11

Função: Consome mensagens do Kafka e indexa no ElasticSearch, adicionando processamento extra.

Processamento Adicional:

1. **Extração de Keywords:**
 - Tokenização do texto (títulos + comentários)

- Remoção de stopwords em inglês
 - Filtro por tamanho mínimo (4 caracteres)
 - Seleção das top 20 palavras mais frequentes
2. **Classificação de Sentimento:**
- Conversão do compound score em label categórico
 - Adição de timestamp UTC
3. **Indexação:**
- Envio para Elasticsearch via API REST
 - Criação automática do índice com mapping apropriado

3.2.7 7. Kibana

Versão: 8.11.0

Função: Interface de visualização e criação de dashboards interativos.

Porta: 5601

Data View: reddit-sentiment (índice do Elasticsearch)

3.3 Visualizações Implementadas

3.3.1 1. Nuvem de Palavras (Tag Cloud)

Tipo: Tag Cloud

Objetivo: Visualizar as palavras mais frequentes nos comentários.

Configuração: - Campo: `keywords` - Size: Top 100 palavras - Tamanho proporcional à frequência

Variações: - Nuvem geral (todas as palavras) - Nuvem de palavras positivas (filtro: `sentiment_label = positive`) - Nuvem de palavras negativas (filtro: `sentiment_label = negative`)

Insights: Identifica os tópicos mais discutidos e as palavras associadas a cada sentimento.

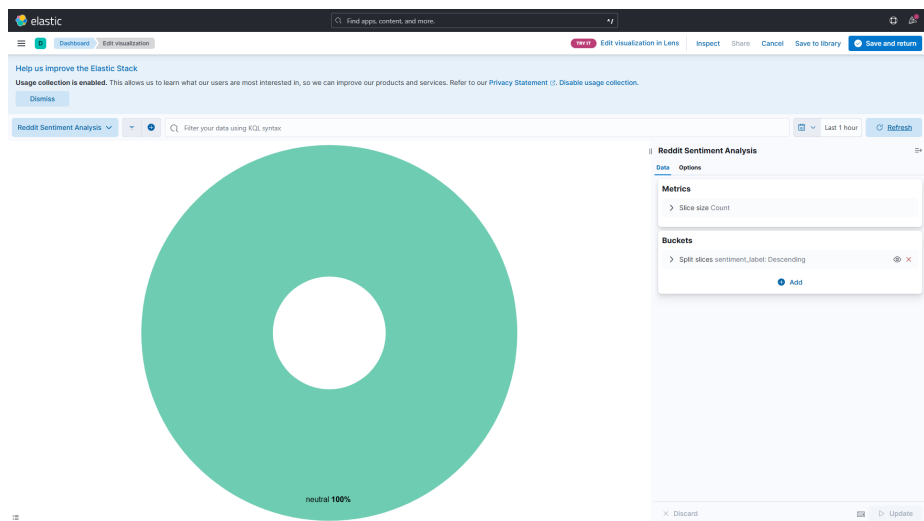
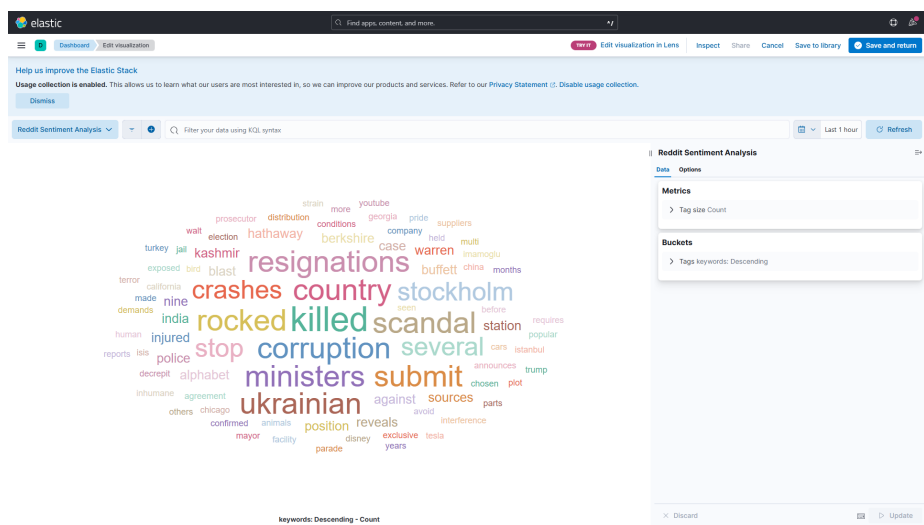
3.3.2 2. Distribuição de Sentimentos (Gráfico de Pizza)

Tipo: Pie Chart

Objetivo: Mostrar a proporção de comentários positivos, negativos e neutros.

Configuração: - Agregação: `Terms` - Campo: `sentiment_label.keyword` - Visualização: 3 fatias coloridas (verde/vermelho/cinza)

Insights: Permite identificar rapidamente o tom geral das discussões no Reddit.



3.3.3 3. Contador de Palavras (Gráfico de Barras Horizontal)

Tipo: Horizontal Bar Chart

Objetivo: Exibir as palavras mais frequentes em ordem decrescente de ocorrência.

Configuração: - Eixo Y: **keywords** (Terms aggregation) - Eixo X: Count (contagem de ocorrências) - Size: Top 20 palavras - Order by: Metric: Count - Order: Descending

Insights: Identifica rapidamente os termos mais discutidos nos comentários do Reddit, permitindo análise de tópicos em destaque. Diferente da nuvem de palavras, apresenta valores numéricos exatos de frequência.

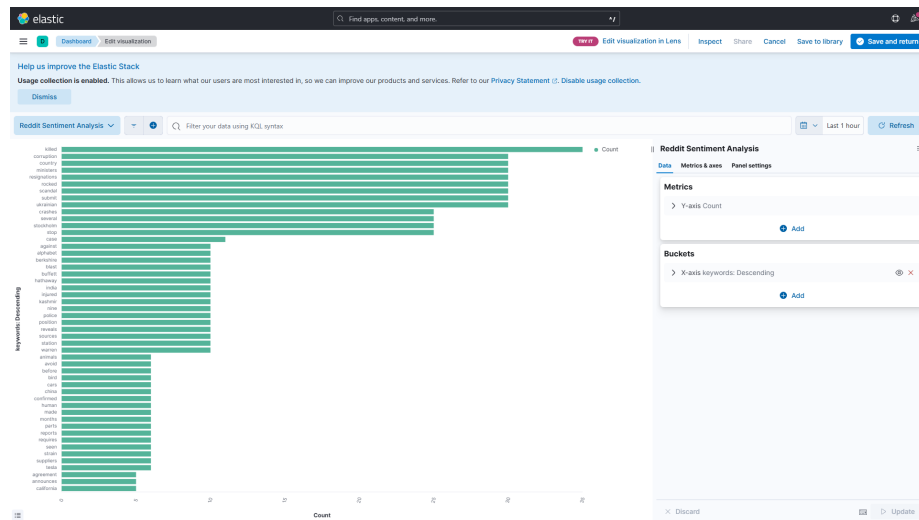


Figure 7: Visualização - Contador de Palavras

3.3.4 4. Sentimentos ao Longo do Tempo (Gráfico de Barras)

Tipo: Vertical Bar Chart

Objetivo: Analisar a evolução temporal dos sentimentos.

Configuração: - Eixo X: **timestamp** (Date Histogram) - Eixo Y: Count - Breakdown: **sentiment_label.keyword** - Visualização: Barras empilhadas por sentimento

Insights: Identifica padrões temporais, picos de sentimentos negativos/positivos relacionados a eventos específicos.

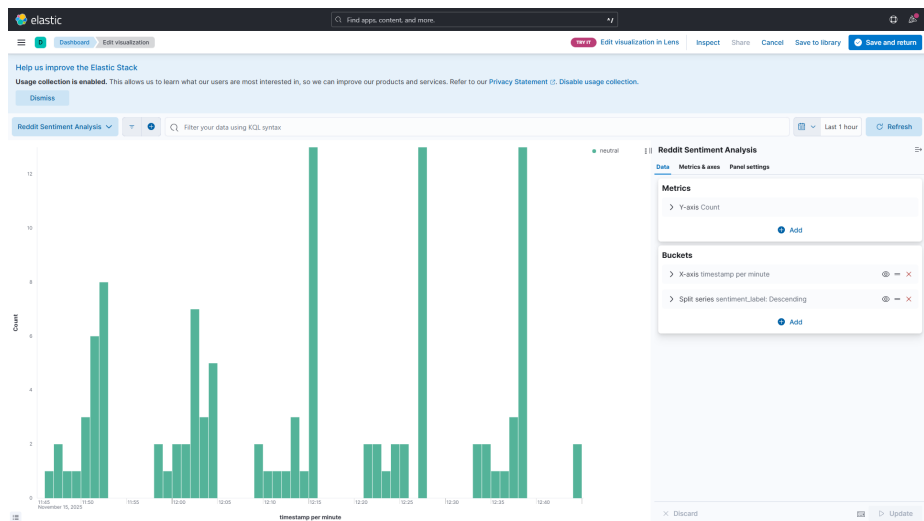


Figure 8: Visualização - Sentimentos ao Longo do Tempo

3.3.5 5. Dashboard Completo

Nome: Reddit Sentiment Dashboard

Componentes: - Todas as visualizações acima organizadas em um layout responsivo - Filtros interativos por período de tempo - Busca por palavras-chave - Auto-refresh configurável

Funcionalidades: - Drill-down: Clicar em elementos para filtrar dados - Time range: Ajustar período de análise - Export: Exportar dados e visualizações - Share: Compartilhar dashboard via URL

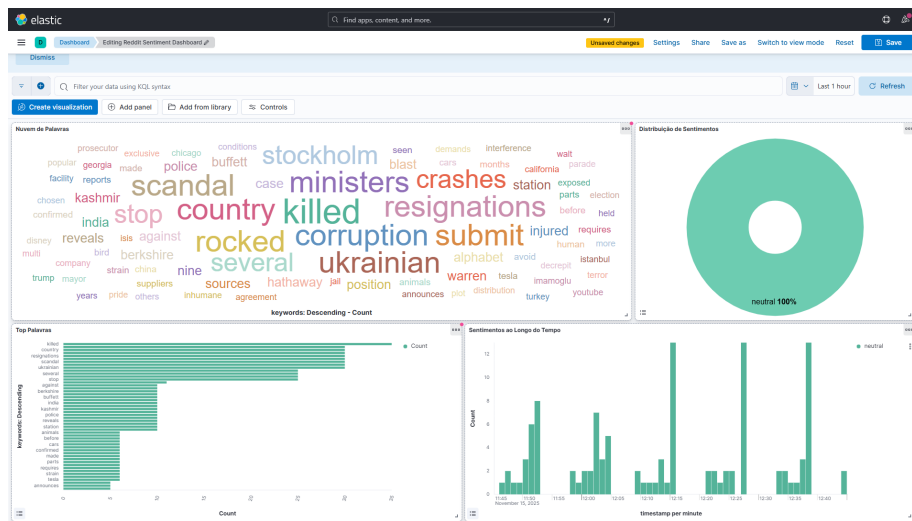
3.4 Como Executar a Aplicação

3.4.1 Pré-requisitos

- **Docker Desktop** instalado e rodando
- **Docker Compose** (incluído no Docker Desktop)
- Mínimo **4GB de RAM** disponível para o Docker
- Portas disponíveis: 8888, 5601, 9200, 9092, 7077, 8080, 2181

3.4.2 Primeira Execução - Passo a Passo Detalhado

```
1 git clone https://github.com/dev-brito/t2-pspd-2025_2.git
2 cd spark
```



```
1 docker compose up --build
```

O que acontece: - Download das imagens Docker (a primeira vez demorará alguns minutos) - Build das imagens customizadas (Spark, Scraper, Consumer) - Inicialização dos containers na ordem correta: 1. Zookeeper 2. Kafka 3. ElasticSearch 4. Spark Master e Worker 5. Kibana 6. Jupyter Notebook (Spark) 7. Reddit Scraper 8. Consumer Kafka-to-ElasticSearch

Verificar se está rodando:

```
1 docker ps
```

Você deve ver 8 containers rodando.

3.4.2.3 3. Acessar o Jupyter Notebook URL: <http://localhost:8888>

Obter Token de Autenticação:

Entre nos logs do container `spark-notebook` e procure por `http://127.0.0.1:8888/lab?token=`

Copie o token e cole na página de login, ou use a URL completa com token.

Abrir o Notebook: Navegue até `spark_notebook.ipynb`

3.4.2.4 4. Executar o Processamento Spark No Jupyter Notebook, execute as células em ordem, exceto a última.

Resultado: O Spark Streaming começa a processar dados continuamente.

3.4.2.5 5. Verificar Dados no Elasticsearch Aguarde 2-3 minutos para dados serem coletados e processados.

Verificar contagem de documentos:

```
1 curl http://localhost:9200/reddit-sentiment/_count
```

Resultado esperado:

```
1 {"count":50,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0}}
```

Buscar documentos de exemplo:

```
1 curl http://localhost:9200/reddit-sentiment/_search?size=3&pretty
```

3.4.2.6 6. Configurar Kibana Acessar Kibana: <http://localhost:5601>

3.4.2.6.1 Importar Data View e Visualizações Para facilitar a configuração, o projeto inclui um arquivo com todas as visualizações pré-configuradas.

Passos para Importação:

1. **Acessar Stack Management:**
 - Menu lateral → **Management** → **Stack Management**
2. **Navegar até Saved Objects:**
 - No menu lateral de Stack Management → Seção **Kibana** → **Saved Objects**
3. **Importar o arquivo:**
 - Clique em **Import**
 - Faça upload do arquivo `spark/kibana-export.ndjson`
 - Clique em **Import** para confirmar
4. **Acessar o Dashboard:**
 - Após a importação, clique em **Reddit Sentiment Dashboard**
 - Todas as visualizações estarão configuradas e prontas para uso

3.4.3 Execuções Subsequentes

Após a primeira configuração, para executar novamente:

```
1 # Iniciar containers
2 docker compose up -d
3
4 # Acessar Jupyter e executar células do notebook
5 # URL: http://localhost:8888
6
```



```
7 # Acessar Kibana (visualizações já estarão salvas)
8 # URL: http://localhost:5601
```

3.4.4 Parar a Aplicação

```
1 # Parar containers
2 docker compose down
3
4 # Parar e remover volumes (limpar dados)
5 docker compose down -v
```

3.5 Conclusão sobre o Uso do Spark

O Apache Spark demonstrou ser a ferramenta ideal para este projeto de análise de sentimentos em tempo real. Sua capacidade de processar streams de dados com baixa latência, combinada com uma API de alto nível e integração nativa com Kafka e Elasticsearch, permitiu criar um pipeline completo e funcional com código conciso e manutenível.

O projeto demonstra como o Spark pode ser utilizado em cenários práticos de análise de dados de redes sociais, fornecendo insights valiosos através de visualizações interativas no Kibana. A arquitetura implementada é robusta, escalável e serve como base sólida para expansões futuras, como análise de múltiplas redes sociais, modelos de machine learning mais sofisticados, ou processamento de volumes massivos de dados em ambiente de produção.

3.5.1 Dificuldades encontradas

Uma das primeiras dificuldades encontradas foi conseguir alterar a fonte de dados para o `input_topic` de modo a utilizar alguma rede social. Primeiro, tentamos integrar um bot do Discord, mas não conseguimos fazê-lo enviar mensagens de forma consistente para o Kafka. Em seguida, tentamos utilizar a API/web do X (Twitter), porém fomos impedido por bloqueios e restrições do próprio site. Por fim, recorremos ao `old.reddit`, que também apresentou obstáculos — principalmente bloqueios de acesso (403) e necessidade de adaptar o scraper ao HTML irregular e às limitações impostas pelo site.

Outra dificuldade foi ao tentar realizar todo o pipeline pelo próprio Google Colab, o que se mostrou altamente difícil, senão impossível. Portanto, foi necessário containerizar a solução, deixando apenas a parte do processamento dos dados vindos da fila pelo Spark Streaming dentro de um notebook, que, em seguida, envia os dados para outra fila do Kafka.

Depois que o scraper começou a funcionar e alimentar o `input_topic`, enfrentamos uma sequência de dificuldades técnicas profundas ao tentar integrar o Spark Structured Streaming 4.0.1 com o Kafka. Primeiro, a resolução automática de dependências com `.config("spark.jars.packages", ...)`

falhava porque as versões mais novas do Kafka não possuem compatibilidade binária com Spark 4.0.1, resultando em erros como `NoClassDefFoundError` e `JavaGatewayExited`. Tentamos diversas combinações de jars no Dockerfile, mas isso gerou novos conflitos, como ausência de classes (`GenericKeyedObjectPoolConfig`) ou métodos (`setMinEvictableIdleDuration()`). Isso levou a uma série de erros silenciosos, micro-batches abortados e ausência de mensagens no `output_topic` até que, finalmente, após retornar ao método de config diretamente no notebook, todas as dependências foram alinhadas. Somado aos problemas de checkpoint, offsets antigos e comportamento do Spark com topics vazios, o processo exigiu inúmeras reconstruções de containers e ajustes finos para estabilizar todo o pipeline.

4 Autoavaliação

- **Ana Luíza Rodrigues:** Durante o desenvolvimento do trabalho fui responsável pela montagem do cluster Hadoop utilizando Docker Compose. Enfrentei dificuldades na execução de scripts Python via Hadoop Streaming, pois as imagens Docker não possuíam Python3 instalado e os repositórios Debian Stretch estavam descontinuados. Na parte do Spark, auxiliei na validação e testes do pipeline de streaming, contribuindo com a verificação do fluxo de dados entre Kafka e Elasticsearch e auxiliando na documentação das visualizações do Kibana. Esse laboratório foi muito enriquecedor pois nunca tinha utilizado o Hadoop nem o Apache Kafka. Tive dificuldade em utilizar os frameworks, mas a integração do grupo no desenvolvimento ajudou bastante.
- **Guilherme Brito Vilas Boas:** Neste trabalho, minha atuação principal concentrou-se no estudo e implementação de testes de performance e tolerância a falhas no framework Hadoop. A fase inicial foi desafiadora, pois a montagem do cluster em Docker exigiu a identificação e alteração de uma configuração específica, o que demandou alguns dias de análise. Ao final, foi gratificante comprovar a resiliência do Hadoop, que manteve a execução dos jobs com sucesso mesmo com a perda de nodes. Na parte do Spark, contribuí com a análise e validação dos resultados do processamento de sentimentos, auxiliando na interpretação dos dados processados e na verificação da consistência das análises realizadas pelo pipeline de streaming.
- **Laís Ramos Barbosa:** No desenvolvimento do trabalho atuei nos testes relativos às configurações do Hadoop e auxiliando nas configurações gerais do Ambiente Spark e Kafka. Participei ativamente da configuração inicial do ambiente Spark, incluindo a integração com Kafka e a validação do fluxo de dados no pipeline de streaming. Pra mim foi muito enriquecedor aprender a configurar e aplicar essas tecnologias, visto que são tecnologias amplamente usadas no mercado. Já havia usado o Apache Kafka no trabalho, mas os outros frameworks foram a primeira vez. Foi desafiador,

mas o grupo soube dividir bem as tarefas e sempre que um integrante tinha dúvidas, o outro estava disponível para auxiliar.

- **Lucas Lopes Rocha:** Fui responsável pela integração do Elasticsearch e Kibana ao pipeline de streaming. Implementei o consumer `Kafka-to-ElasticSearch` com extração automática de palavras-chave e classificação de sentimentos, criei as visualizações no Kibana e desenvolvi a documentação da parte do Spark. Na parte do Hadoop, auxiliei na validação das configurações do cluster, participei dos testes de execução do WordCount e contribuí com a análise dos resultados dos testes de tolerância a falhas. Enfrentei desafios com compatibilidade de versões da biblioteca `elasticsearch-py`, resolvidos através de ajustes nos parâmetros de conexão e na habituação inicial com o Kibana para criar as visualizações. A experiência com tecnologias de Big Data (Spark, Kafka, Elasticsearch, Kibana, Hadoop) foi extremamente enriquecedora, proporcionando compreensão prática de arquiteturas de streaming modernas e preparação para desafios profissionais em engenharia e análise de dados em tempo real.
- **Matheus Raphael Soares de Oliveira:** No desenvolvimento do trabalho atuei principalmente na parte de configuração do Kafka, nos tópicos de input e output, programando o scraper do Reddit, assim como nas configurações e análise de sentimentos do Spark Streaming. No Hadoop, auxiliei nas configurações gerais do cluster junto com meus outros colegas, participei da execução e validação dos testes de MapReduce, e contribuí com a análise dos logs e comportamento do sistema durante os testes de tolerância a falhas. Aprendi muito sobre Kafka e Spark na execução desse trabalho, além de afinar meus conhecimentos sobre Docker e Docker compose. Consegui perceber que o Spark é uma tecnologia que facilita bastante o processamento de dados de streaming, possibilitando análise em tempo real de grandes volumes de dados. Foi desafiador, encontrei bastante erros, mas no final, com a ajuda do grupo, conseguimos deixar essa pipeline de análise de dados em fila funcionando.